Database Course

Views and Materialized Views in SQL

Lotfi NAJDI

Polydisciplinary Faculty of Taroudant

2025-2026

Database Objects

- In a database, we have various objects to organize and manage data:
- you already know about tables but there are many others, including:

```
- Views
- Materialized Views
- Indexes
- Sequences
- Functions
- Triggers
- Stored Procedures
- Schemas
- ...
```

What is a View?

A view is a **stored query** that you can interact with as if it were a real table.

It's a **"virtual table"** 📋

No data is actually stored in the view itself

A Simple Analogy 🖥️

Think of a view as a window to look through to see specific data from one or more tables.

- The window doesn’t contain the actual data
- It just provides a simple way to access it
- Click the window ⟶ opens the real data
- Query the view ⟶ runs the real query

So, Why Use Views? 🤔

Views have **three superpowers**:

🧠 **Simplicity** - Hide complex queries

🛡️ **Security** - Control data access

🔄 **Consistency** - Provide stable interfaces and Create logical data models independent of physical schema

Superpower #1: Simplifying Complexity 👨‍💻

The Problem:

```sql
-- This complex query joins 5 tables!
SELECT c.customer_name,
       SUM(p.amount) as total_spent,
       COUNT(o.order_id) as order_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN payments p ON o.order_id = p.order_id
WHERE p.payment_date >= '2023-01-01'
GROUP BY c.customer_id, c.customer_name;
```

Superpower #1: The Solution ✨

Create a view:

```sql
CREATE VIEW customer_summary AS
SELECT c.customer_name,
       SUM(p.amount) as total_spent,
       COUNT(o.order_id) as order_count
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN payments p ON o.order_id = p.order_id
WHERE p.payment_date >= '2023-01-01'
GROUP BY c.customer_id, c.customer_name;
```

Now it’s simple:

```sql
SELECT * FROM customer_summary;
```

Querying the View

After creating the view, you can query it like a regular table!

```sql
SELECT * FROM customer_summary;
```

customer_name	total_spent	order_count
Amine	1500.00	5
Karima	2300.00	8
Hasna	1200.00	3
Ahmed	1800.00	6

Superpower #2: Enhancing Security 🛡️

The Problem: - An intern needs employee names and start dates - But must NOT see salaries or addresses - The `employees` table has sensitive data

The Solution:

```sql
CREATE VIEW employee_anniversaries AS
SELECT name, start_date
FROM employees;
```

A view acts like a curtain, showing only what you choose 🎭

Superpower #3: Maintaining Consistency 🔄

- Databases are not static. They must evolve for performance, normalization, or new features.
- This often means complex changes, like **decomposing** a single large `users` table into multiple, specialized tables.

The Challenge: - Any direct change to the underlying tables **will break** every application, report, and query that depends on them.

Superpower #3: Maintaining Consistency 🔄

Before: A single, large table

```
[ users table ]
  - user_id
  - primary_email
  - order_id
  - order_date
  - total_amount
```

After: Decomposed tables

```
[ users table ]
  - user_id
  - primary_email
[ orders table ]
    - order_id
    - user_id
    - order_date
    total_amount
```

Superpower #3: Maintaining Consistency 🔄

The Solution: A Stable "API" 💡

An SQL `VIEW` provides a powerful **abstraction layer**. It acts like a virtual table, shielding applications from backend complexity.

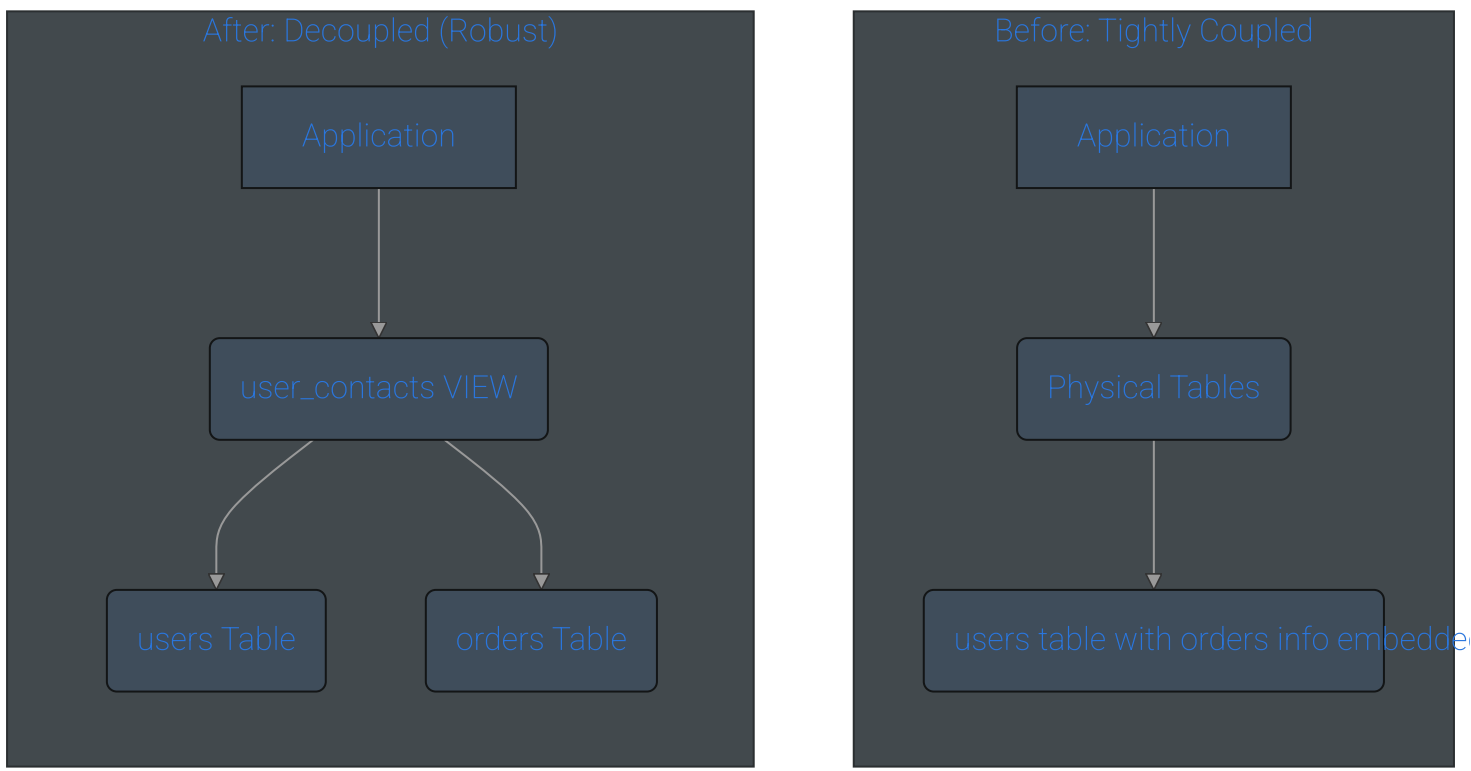
- Instead of querying tables directly, your applications query the view.
- If the underlying tables change, you only need to update the view's logic.
- The application's code doesn't change at all.

The Code:

```sql
CREATE VIEW user_contacts AS
SELECT
  u.primary_email AS email,
  o.order_id,
  o.order_date,
  o.total_amount
FROM users u
JOIN orders o
  ON u.user_id = o.user_id;
```

The Benefit: A Stable Contract 🤝

- Views decouple your application logic from your physical data storage. - You gain the freedom to optimize and refactor your database schema without causing downstream chaos.



The “How”: Creating a View 

Basic Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example:

```
CREATE VIEW active_customers AS
SELECT customer_id, customer_name, email
FROM customers
WHERE status = 'active';
```

🎯 Exercise: Your First View

Using a `products` table, write a command to create a view named `kitchen_gadgets` that shows the `name` and `price` of products in the 'Kitchen' category.

Answer:

```sql
CREATE VIEW kitchen_gadgets AS
SELECT name, price
FROM products
WHERE category = 'Kitchen';
```

Managing Views: Modifying 🔧

The best way to change a view:

```
CREATE OR REPLACE VIEW view_name AS
SELECT new_columns...
FROM table_name;
```

Why use OR REPLACE? - Updates the view without dropping it first - Preserves permissions and dependencies - Safer than DROP + CREATE

Managing Views: Deleting 🗑️

To delete a view:

```
1  DROP VIEW view_name;
```

For safety, use:

```
1  DROP VIEW IF EXISTS view_name;
```

Avoids errors if the view doesn’t exist

Confirming Creation in the System 🔍

Where do views live?

PostgreSQL stores view information in the `pg_views` system catalog.

```
1 SELECT viewname, definition
2 FROM pg_views
3 WHERE schemaname = 'public';
```

viewname	definition
customer_summary	SELECT c.customer_name, SUM(p.amount) …
employee_anniversaries	SELECT name, start_date FROM employees;
user_contacts	SELECT primary_email AS email, phone FROM users;

What if Your View is Very Slow? 🐌

The Problem:

```sql
-- This complex report takes 2 minutes every time!
CREATE VIEW monthly_report AS
SELECT ... -- Complex aggregations across millions of rows
FROM large_table1 t1
JOIN large_table2 t2 ON ...
WHERE ... -- Complex conditions
GROUP BY ... -- Heavy grouping
```

Too slow for a user-facing dashboard ⏰

The Solution: Materialized Views 📸

A materialized view physically stores its results

Analogy:

🖼️ Regular view = Live window (*always shows current scene*)

📸 Materialized view = Photograph (*frozen moment in time*)

The Core Trade-Off ⚖️

Performance vs. Freshness

📸 The photograph is **super fast** to look at

⏰ But it gets **out of date**

🎯 You must choose: High performance OR real-time data

Creating a Materialized View 

Syntax:

```
CREATE MATERIALIZED VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example:

```
CREATE MATERIALIZED VIEW monthly_sales_summary AS
SELECT
    DATE_TRUNC('month', sale_date) as month,
    COUNT(*) as total_sales,
    SUM(amount) as total_revenue
FROM sales
GROUP BY DATE_TRUNC('month', sale_date);
```

Refreshing a Materialized View 🔄

The data does NOT update automatically!

You must run this command:

```
1  REFRESH MATERIALIZED VIEW view_name;
```

Example:

```
1  REFRESH MATERIALIZED VIEW monthly_sales_summary;
```

Think of it as taking a new photograph 📸

Confirming a Materialized View 📋

System catalog for materialized views:

```
1 SELECT matviewname, ispopulated
2 FROM pg_matviews
3 WHERE schemaname = 'public';
```

What is `ispopulated`?

- `true` = View has data (photograph taken)
- `false` = View is empty (no photograph yet)

Execpect output:

matviewname	ispopulated
monthly_sales_summary	true

When is a Materialized View Empty? 🚫

Initial Creation without Data: A materialized view is created empty when the WITH NO DATA clause is explicitly used (e.g., CREATE MATERIALIZED VIEW mv_name AS SELECT … WITH NO DATA;)

Explicit Refresh to Empty: The view becomes empty when a user deliberately runs the command REFRESH MATERIALIZED VIEW mv_name WITH NO DATA;, which discards any existing data without re-running the view’s query.

Advanced Topic: Updatable Views 

Can you INSERT into a view?

Sometimes! This is called an “updatable view.”

The Golden Rule: *It only works if PostgreSQL can unambiguously find the single row in the underlying table to change.*

Regular View vs. Materialized View

Feature	Regular View (Virtual)	Materialized View (Snapshot)
Data Storage	**No** – Stores only the query definition.	**Yes** – Stores pre-calculated data (a snapshot).
Data Freshness	**Always real-time** – Query executed on demand.	**Potentially stale** – Data is current as of its last refresh.
Performance	**Slower** for complex queries (re-executes every time).	**Faster** for reads (data is pre-computed).
Update Mechanism	No explicit update/refresh needed (always current).	Requires `REFRESH MATERIALIZED VIEW` to update its data.
Storage Cost	Very low (only query text).	Higher (stores a copy of the data).
Best For	Real-time data, security, simplifying complex queries.	Reporting, analytics, reducing load on source tables, faster reads.

Updatable Views: The Basics

- We can perform INSERT, UPDATE, DELETE operations on a view if it meets certain criteria.
- But remember we are not updating the view itself, we are updating the underlying base table through the view.

Scenario: Updatable View

1. Security-restricted edits (allow safe updates to non-sensitive fields)

```sql
-- Create a simple view exposing only non-sensitive columns
CREATE VIEW staff_hire_dates AS
SELECT id, full_name, hire_date
FROM staff;
```

```sql
-- Update via the view (updates underlying table)
UPDATE staff_hire_dates
SET hire_date = '2022-05-01'
WHERE id = 42;
```

Scenario: Updatable View

2. Department-scoped inserts (enforce tenant/department constraints) – View restricts rows to a single category and enforces it on writes CREATE VIEW gadgets_kitchen AS SELECT id, name, price, category FROM products WHERE category = 'Kitchen' WITH LOCAL CHECK OPTION;

– Insert through the view; CHECK OPTION prevents creating rows outside 'Kitchen' INSERT INTO gadgets_kitchen (name, price, category) VALUES ('Compact Citrus Zester', 12.50, 'Kitchen');

The Rules for "Simple" Views ✅

What makes a view updatable?

The view must be **"simple":**

- ✅ Selects from only ONE table (no JOINs)
- ✅ No aggregations (COUNT, SUM), GROUP BY, or DISTINCT
- ✅ No window functions or complex expressions

🎯 Exercise: Which is Updatable?

Question: Which of these views is updatable?

View A:

```sql
CREATE VIEW electronics AS
SELECT * FROM products
WHERE category = 'Electronics';
```

View B:

```sql
CREATE VIEW category_prices AS
SELECT category, AVG(price)
FROM products
GROUP BY category;
```

Answer: View A is updatable! (Simple, single table, no aggregations)

🎯 Interactive Exercise: Which is Updatable?

Question: Which of these views is updatable? **View A:**

```sql
CREATE VIEW electronics AS
SELECT DISTINCT  price, color
FROM products
WHERE category = 'Electronics';
```

View B:

```sql
CREATE VIEW product_orders AS
SELECT p.product_id,
       p.name       AS product_name,
       o.order_id,
       o.order_date,
       oi.quantity,
       c.customer_name
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o       ON oi.order_id = o.order_id
JOIN customers c    ON o.customer_id = c.customer_id;
```

The CHECK OPTION Solution

Enforcing the view's rules:

```sql
CREATE VIEW west_region_customers AS
SELECT customer_id, name, region, address, status, credit_limit
FROM customers
WHERE region = 'West'
WITH LOCAL CHECK OPTION;
```

```sql
-- Allowed: region matches the view's WHERE clau
INSERT INTO west_region_customers (customer_id,
VALUES (1001, 'Acme West', 'West', '123 Pacific
```

```sql
-- Success: Row inserted
```

```sql
-- Rejected: region does not match -> fails due
INSERT INTO west_region_customers (customer_id,
VALUES (1002, 'Acme East', 'East', '10 Atlantic
```

```sql
-- Error: New row violates view's CHECK OPTION
```

Summary 📚

Views: Virtual shortcuts for simplicity, security, and consistency

Materialized Views: Physical snapshots for high performance

Advanced: Simple views can be updated, CHECK OPTION protects data integrity

Key Takeaway: Choose the right tool for your specific use case! 🎯